

CS 4782 Extra Credit Demo Rationale
Automatic Differentiation (Computational Graphs)
NetID: rx77

I chose automatic differentiation because the course website's backprop explanation is a dense read and hard to visualize. I was also personally curious how libraries like PyTorch compute derivatives quickly without relying on numerical differentiation, so I built a demo that makes the algorithmic process concrete while showing why it is efficient. The demo centers the computation graph first to make the visual flow of values and gradients the primary learning experience.

The design emphasizes clarity and engagement. Students can build a graph from an expression, run forward and backward passes, and step through each node with a terminal-style log. This makes the chain rule feel procedural rather than abstract. I included common ML functions (log, exp, tanh, sigmoid/softplus, etc.) and loss terms so the examples feel familiar to students who have seen neural networks. The forward-mode tab shows dual numbers side-by-side with a numerical check to build trust in the derivatives, while the gradient descent tab illustrates how those gradients drive learning and why the learning rate can look stable early but still overshoot and explode later if it is too large.

I also contrasted graph-based differentiation with numerical intuition by discussing numerical differentiation in the context of precision and overhead. A next improvement (already identified in the demo rationale) is to add explicit timing to compare numerical vs. graph-based methods. Overall, the demo focuses on the “why” (efficiency and correctness) and the “how” (graph traversal and local derivatives), with an interface that is easy to use in-class.